Loan Approval Prediction: End-to-End Machine Learning Project

Prepared by: Noor ul Aien

Organization: Devixo Solutions

Internship Domain: AI/ML

Internship Task: Task 03 – End-to-End Machine Learning Project

Devixo Solutions Summer Internship | AI/ML Internship | Task 03 | Report Date: 8 November 2026

1. Introduction and Objective

The goal of this project was to build a complete machine learning solution, from raw data to a deployed prediction interface. The project covers data preparation, feature engineering, feature selection, scaling, training of multiple models, cross validation, hyperparameter tuning, feature importance analysis, model saving with Joblib and Pickle, documentation, and a Streamlit app as a bonus.

The chosen problem is loan approval prediction: given an applicant's financial profile, predict whether a bank would approve or reject the loan. This is a binary classification task.

Tools used: Python, Pandas, NumPy, Scikit-learn, XGBoost, Matplotlib, Seaborn, Joblib, Pickle, Streamlit, Google Colab and VS Code.

2. Dataset

Item	Detail
Name	Loan Approval Prediction Dataset (Kaggle, by Archit Sharma)
File	loan_approval_dataset.csv
Size	4,269 rows and 13 columns (the task requires at least 3,000 rows)
Target	loan_status (Approved = 1, Rejected = 0)
Class balance	62.2% Approved, 37.8% Rejected
Missing values	0
Duplicates	0

The input columns are number of dependents, education, self-employment status, annual income, loan amount, loan term, CIBIL score, and four asset values (residential, commercial, luxury and bank). The identifier column loan_id carries no predictive information and was dropped.

A data quality issue found early: the raw column names and text values contained leading spaces (for example " loan_status"). These were stripped first, otherwise later steps would fail with key errors.

3. Workflow

3.1 Data Preparation

- Imported the CSV and inspected shape, types and column names.
- Stripped extra spaces from column names and text values.
- Checked missing values (none found) and duplicates (none found).
- Dropped loan_id and encoded the target as 1 and 0.
- Applied a median imputer inside the pipeline so the process stays reusable if future data has missing values.

3.2 Feature Engineering

Categorical columns were encoded (education: Graduate = 1; self_employed: Yes = 1). Five new features were created using ordinary banking logic:

New feature	Formula	Meaning
total_assets	sum of the four asset values	Overall wealth of the applicant
loan_to_income	loan_amount / income_annum	How large the loan is compared with income
assets_to_loan	total_assets / loan_amount	How well assets cover the loan
income_per_dependent	income_annum / (dependents + 1)	Income available per family member
monthly_payment	loan_amount / (loan_term x 12)	Approximate monthly repayment burden

This raised the dataset from 12 to 17 columns.

3.3 Correlation Analysis

The correlation heatmap showed that cibil_score has by far the strongest relationship with loan_status (0.77). Other features were only weakly correlated with the target, for example loan_term (-0.11) and loan_to_income (0.09). The heatmap also showed strong multicollinearity between income_annum,

loan_amount, luxury_assets_value and total_assets (0.85 to 0.93).



3.4 Train/Test Split

The data was split 80/20 with stratification on the target and a fixed random seed (42), giving 3,415 training rows and 854 test rows. The split was done before scaling to avoid data leakage.

3.5 Feature Selection

SelectKBest with the ANOVA F-test (f_classif) and k = 10 was used. The top-scoring feature was cibil_score (score about 4,852), far ahead of monthly_payment (49), loan_term (40) and loan_to_income (32). The 10 selected features were:

no_of_dependents, income_annum, loan_amount, loan_term, cibil_score, luxury_assets_value, total_assets, loan_to_income, assets_to_loan, monthly_payment.

Because k was fixed at 10, a few low-scoring features (such as luxury_assets_value and total_assets) were kept. Tree-based models can still extract information from them.

3.6 Scaling

StandardScaler was fitted on the training data only and applied to both sets, giving every feature a mean of 0 and a standard deviation of 1. Scaled data was used for the SVM. Tree-based models do not need scaling and used the original values.

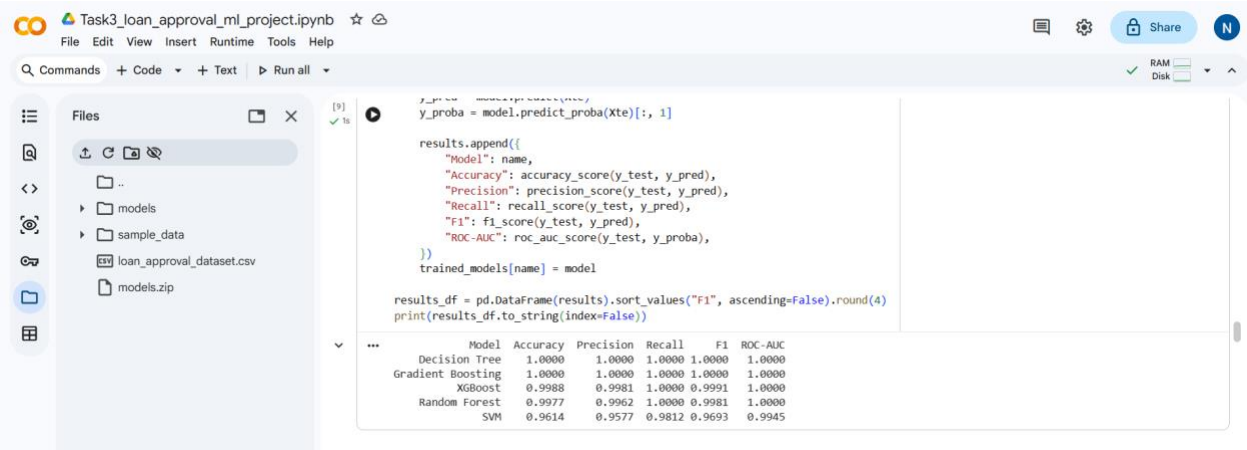
4. Model Development

Five models were trained, which exceeds the minimum of four: Random Forest, Decision Tree, Gradient Boosting, Support Vector Machine and XGBoost (optional in the task). Each was evaluated on the test set with Accuracy, Precision, Recall, F1 and ROC-AUC.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Decision Tree	1.0000	1.0000	1.0000	1.0000	1.0000
Gradient Boosting	1.0000	1.0000	1.0000	1.0000	1.0000
XGBoost	0.9988	0.9981	1.0000	0.9991	1.0000
Random Forest	0.9977	0.9962	1.0000	0.9981	1.0000
SVM	0.9614	0.9577	0.9812	0.9693	0.9945

Investigating the near-perfect scores

Perfect scores are unusual, so they were investigated rather than accepted. Target leakage was ruled out because the target is not part of any feature. A decision tree with a single split on cibil_score alone (the rule: reject if the score is at most 549.5, otherwise approve) already reaches 96.72% test accuracy. The full Decision Tree gives cibil_score an importance of 0.81, with loan_to_income and loan_term adding the remainder. The conclusion is that the dataset follows a very simple rule dominated by CIBIL score, so high accuracy reflects the dataset, not a flaw in the method.



5. Model Optimization

5.1 Cross Validation

Stratified 5-fold cross validation (F1 score) confirmed that the results are stable and not the product of a lucky split.

Model	CV mean F1	CV std
Decision Tree	1.0000	0.0000
Gradient Boosting	0.9998	0.0005
Random Forest	0.9986	0.0009
XGBoost	0.9984	0.0016

Model	CV mean F1	CV std
SVM	0.9656	0.0069

5.2 Hyperparameter Tuning

GridSearchCV with 5-fold stratified cross validation and F1 scoring was applied to Random Forest, Gradient Boosting, Decision Tree and SVM. XGBoost was left untuned because it is optional in the task.

Model	Best parameters	Best CV F1	Test F1	Test accuracy
Gradient Boosting	learning_rate 0.05, max_depth 3, n_estimators 100	0.9998	1.0000	1.0000
Decision Tree	max_depth 5, min_samples_split 2	1.0000	1.0000	1.0000
Random Forest	max_depth None, min_samples_split 2, n_estimators 200	0.9988	0.9991	0.9988
SVM	C 10, kernel rbf	0.9729	0.9804	0.9754

Tuning helped the SVM most, raising its F1 from 0.9693 to 0.9804. The other models were already close to perfect.

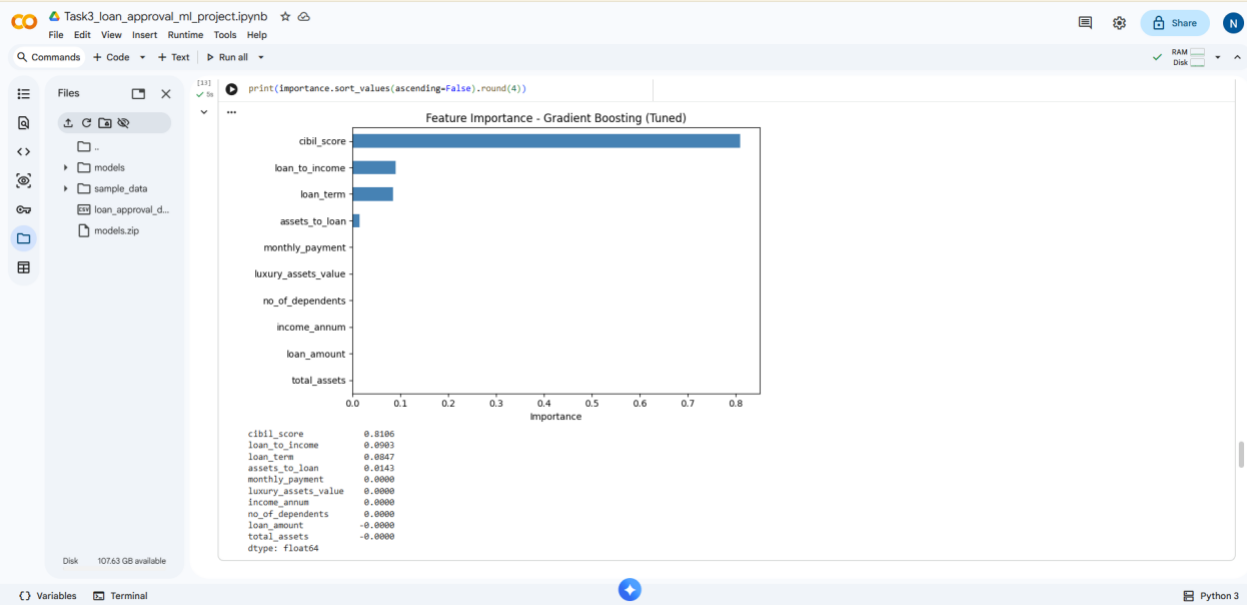
5.3 Feature Importance

The tuned Gradient Boosting model assigned the following importance:

Feature	Importance
cibil_score	0.8106
loan_to_income	0.0903
loan_term	0.0847
assets_to_loan	0.0143
All other selected features	about 0

In plain terms, CIBIL score drives about 81% of the decision. Two engineered features, loan_to_income and assets_to_loan, contributed, which shows that the feature engineering step added real value, while

raw income and asset columns added almost nothing on their own.



6. Model Selection

Decision Tree and Gradient Boosting both reached a test F1 of 1.0. **Gradient Boosting was selected as the final model** for these reasons:

- An ensemble is more stable than a single tree, which can change noticeably with small changes in the data.
- Its cross validation standard deviation was very small (0.0005).
- It provides well-calibrated probabilities, which the app uses to show an approval probability.

7. Saving the Model

The best model was saved in both formats required by the task:

- models/best_model.joblib (Joblib)
- models/best_model.pkl (Pickle)

The fitted scaler (scaler.joblib) and the list of selected features (selected_features.joblib) were saved as well, so the app builds inputs in the same order the model was trained on. Both saved models were reloaded and verified to give the same F1 of 1.0.

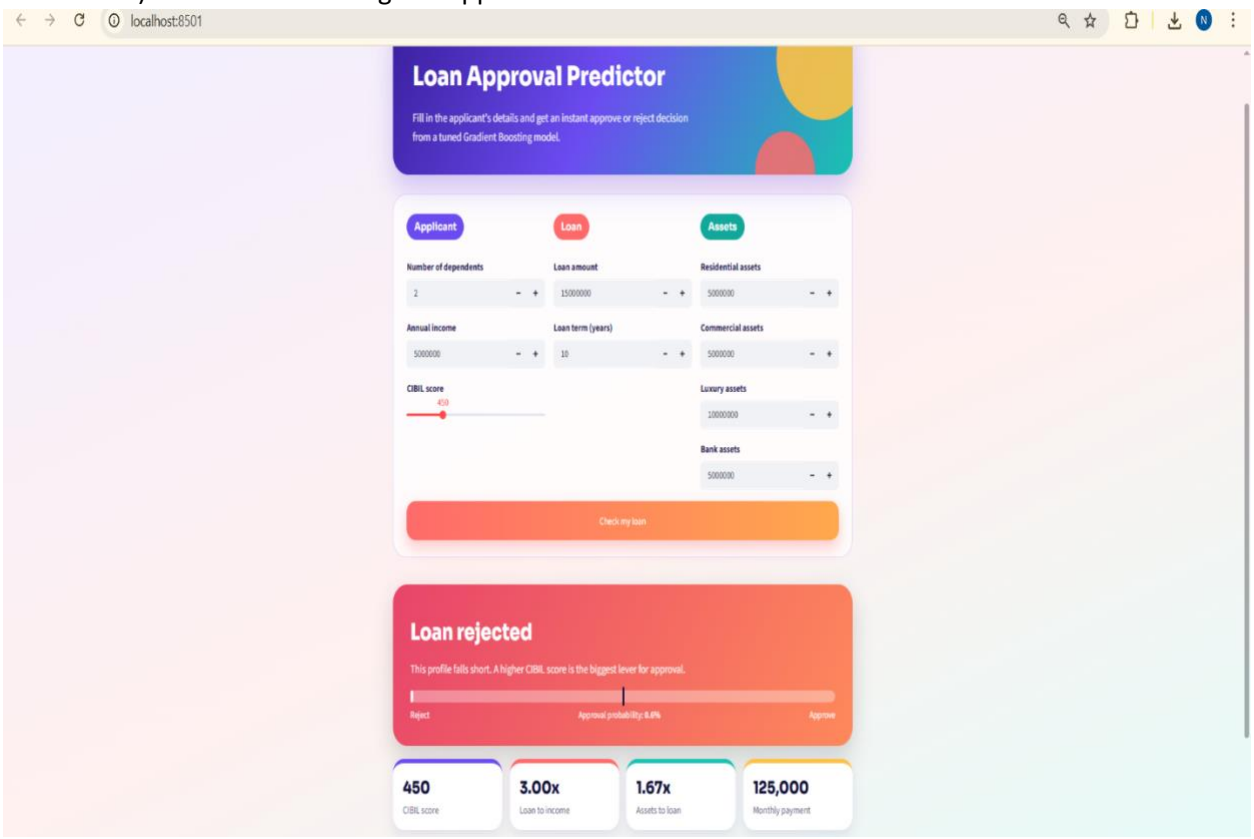
Note on reproducibility: the models first saved in Google Colab could not be loaded on the local machine because of an environment mismatch. The final model was therefore re-trained locally with the script train_and_save.py, using the same data, preprocessing and tuned parameters (scikit-learn 1.6.1), and gives the same test result (F1 and accuracy of 1.0).

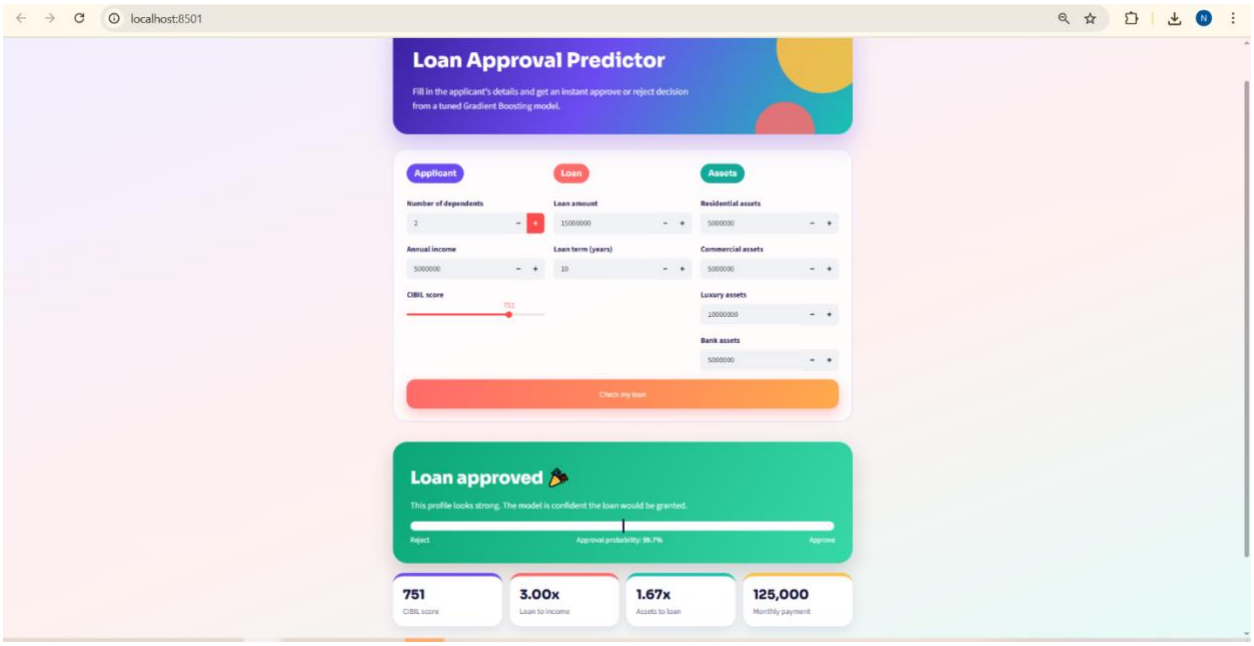
8. Streamlit Prediction App (Bonus)

A Streamlit application (app.py) provides a simple interface. The user enters the applicant's details (dependents, income, CIBIL score, loan amount and term, and four asset values). The app computes the engineered features, runs the saved model and shows:

- an Approved or Rejected result card,
- the approval probability on a meter,
- the key ratios (loan to income, assets to loan, monthly payment).

Testing confirmed the expected behaviour: a CIBIL score of 450 gives Rejected (approval probability about 0.6%) and a score of 750 gives Approved.





To run it: `python -m streamlit run app.py`.

9. Task Requirement Checklist

Requirement	Status
Real-world Kaggle dataset, at least 3,000 rows	Done (4,269 rows)
Import dataset, handle missing values, remove duplicates	Done
Feature engineering, feature selection, data scaling	Done
At least four models (Random Forest, SVM, Gradient Boosting, Decision Tree, XGBoost)	Done (five models)
Hyperparameter tuning, cross validation, feature importance	Done
Save best model with Joblib and Pickle	Done
Documentation (dataset, workflow, model selection, results, limitations, future improvements)	Done
Streamlit interface (bonus)	Done

10. Limitations

- Approval depends almost entirely on CIBIL score (threshold about 550). Near-perfect scores reflect a simple dataset, not necessarily real-world performance.
- The dataset appears synthetic, so results may not transfer to real banking data.
- Several features are highly correlated (income, loan amount, assets), which is multicollinearity.
- SelectKBest with a fixed k of 10 kept a few low-scoring features.

- Class imbalance is mild (62% against 38%) and no resampling was applied.
- XGBoost was trained but not tuned.
- The Streamlit app runs locally and is not deployed online.

11. Future Improvements

- Test the approach on a larger, real-world loan dataset.
- Tune XGBoost and try RandomizedSearchCV or Optuna.
- Handle imbalance with SMOTE or class weights.
- Add model explainability with SHAP.
- Wrap preprocessing and model in a single scikit-learn Pipeline so one file is saved and loaded.
- Pin library versions in requirements.txt and deploy the app on Streamlit Community Cloud.

12. Conclusion

This project delivered a complete machine learning workflow: clean data, engineered features, five trained models, cross-validated and tuned results, a saved best model in two formats, and a working prediction app. Gradient Boosting was chosen as the final model, reaching a test F1 of 1.0. The analysis also showed that CIBIL score dominates this dataset, which is an important point to keep in mind when judging the results.